

Smart Lender

Loan Approval Prediction System — Project Documentation

Project Description

Smart Lender is a Machine Learning–based web application that predicts whether a bank loan application is likely to be approved or declined, based on the applicant's personal and financial details. The system helps banks and financial institutions move faster and more consistently through the loan underwriting process by using historical applicant data to automate the initial approve/decline decision.

The application collects details such as Gender, Marital Status, Number of Dependents, Education, Self-Employment status, Applicant Income, Co-applicant Income, Loan Amount, Loan Term, Credit History, and Property Area. These inputs are passed to a trained Random Forest classification model, which returns an instant Approved or Declined prediction.

Smart Lender is built using Python, Flask, scikit-learn, pandas, and NumPy on the backend, with a custom HTML/CSS front end (styled as a “loan approval ledger”) for data entry and result display. The trained model is serialized with pickle (rdf.pkl) and loaded directly into the Flask application, so predictions are returned in under a second with no external API calls required.

Scenarios

- **Scenario 1:** A loan officer receives an application from a salaried, married applicant with a steady income and a clean credit history. After entering the applicant's details into Smart Lender, the model predicts Approved because the applicant satisfies the key eligibility signals — particularly a positive credit history.
- **Scenario 2:** A self-employed applicant applies for a large loan amount relative to their income, and has no prior credit history on record. Smart Lender evaluates these details and predicts Declined, flagging the application for manual review before any funds are committed.
- **Scenario 3:** A branch processes dozens of loan applications every day. Instead of manually cross-checking each applicant's income, credit history, and dependents against underwriting rules, staff enter the details into Smart Lender and receive an instant approve/decline read, cutting down processing time and keeping decisions consistent across officers.

Technical Architecture

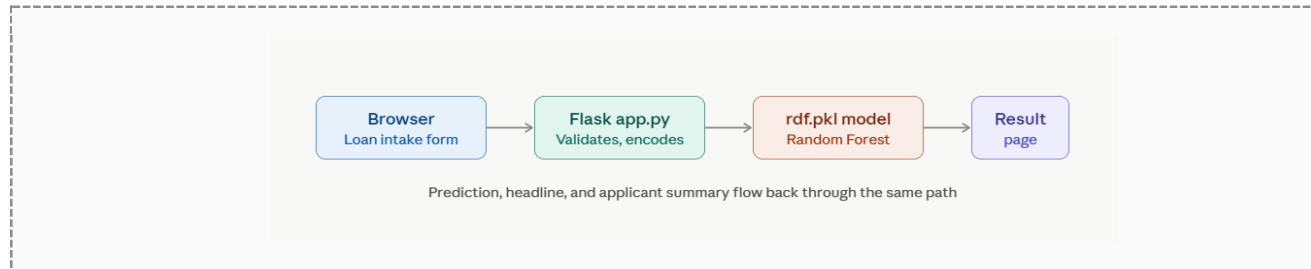


Figure 1: Smart Lender technical architecture diagram

Smart Lender follows a simple three-tier architecture. The presentation layer is a Flask-rendered HTML front end (home, predict, and result pages) where the user enters applicant details. The application layer is the Flask backend (app.py), which validates the submitted form, encodes categorical fields into the same numeric scheme used during training, and calls the trained model. The model layer is a Random Forest classifier (rdf.pkl) trained offline in a Jupyter/Colab notebook and loaded once at server startup. The backend returns the prediction, a short human-readable message, and a summary of the submitted details, which the result page renders back to the user.

Pre-requisites

- Python programming knowledge
- Flask framework
- Machine Learning basics (scikit-learn)
- Pandas & NumPy
- Data visualization (Matplotlib & Seaborn) for EDA
- HTML & CSS
- Git & GitHub
- Google Colab or Jupyter Notebook (for model training)
- VS Code (for the Flask application)
- Loan Prediction dataset (CSV, from Kaggle/Analytics Vidhya)

Project Workflow

The project was built across six milestones:

- Milestone 1: Data Collection and Preprocessing
- Milestone 2: Model Building and Evaluation
- Milestone 3: Flask Backend Development

- Milestone 4: Frontend Development
- Milestone 5: Testing and Deployment
- Milestone 6: Documentation and Conclusion

Milestone 1: Data Collection and Preprocessing

- **Activity 1.1:** Collect the Loan Prediction dataset (Kaggle / Analytics Vidhya “Loan Prediction Problem”) and load it into the Colab notebook with pandas.
- **Activity 1.2:** Explore the dataset: check shape, data types, and class balance of the Loan_Status target using df.info(), df.describe(), and df['Loan_Status'].value_counts().
- **Activity 1.3:** Handle missing values in fields such as Gender, Married, Dependents, Self_Employed, LoanAmount, Loan_Amount_Term, and Credit_History (typically via mode/median imputation).
- **Activity 1.4:** Encode categorical columns to numeric form — Gender, Married, Dependents, Education, Self_Employed, and Property_Area — using the same mapping the Flask app expects (see the GENDER_MAP / MARRIED_MAP / etc. dictionaries in app.py).
- **Activity 1.5:** Split the cleaned data into training and test sets (e.g. train_test_split, 80/20) ready for model building.

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	Credit_History	Property_Area	Loan_Status
0	LP001002	Male	No	0	Graduate	No	5849	0.0	NaN	360.0	1.0	Urban	Y
1	LP001003	Male	Yes	1	Graduate	No	4583	1508.0	128.0	360.0	1.0	Rural	N
2	LP001005	Male	Yes	0	Graduate	Yes	3000	0.0	66.0	360.0	1.0	Urban	Y
3	LP001006	Male	Yes	0	Not Graduate	No	2583	2358.0	120.0	360.0	1.0	Urban	Y
4	LP001008	Male	No	0	Graduate	No	6000	0.0	141.0	360.0	1.0	Urban	Y

```

df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 614 entries, 0 to 613
Data columns (total 13 columns):
 #   Column              Non-Null Count  Dtype
---  -
 0   Loan_ID             614 non-null    object
 1   Gender              601 non-null    object
 2   Married             611 non-null    object
 3   Dependents          599 non-null    object
 4   Education           614 non-null    object
 5   Self_Employed       582 non-null    object
 6   ApplicantIncome     614 non-null    int64
 7   CoapplicantIncome   614 non-null    float64
 8   LoanAmount          592 non-null    float64
 9   Loan_Amount_Term    600 non-null    float64
10   Credit_History       564 non-null    float64
11   Property_Area       614 non-null    object
12   Loan_Status         614 non-null    object
dtypes: float64(4), int64(1), object(8)
memory usage: 62.5+ KB

```

Figure 2: Dataset preview / df.head() and df.info() output

Milestone 2: Model Building and Evaluation

- **Activity 2.1:** Train multiple candidate models on the processed data — e.g. Logistic Regression, Decision Tree, and Random Forest — using scikit-learn.
- **Activity 2.2:** Compare model performance using Accuracy, Precision, Recall, F1-score, and a confusion matrix on the held-out test set.
- **Activity 2.3:** Select the best-performing model. Smart Lender uses a RandomForestClassifier, saved as model/rdf.pkl.
- **Activity 2.4:** Serialize the trained model with pickle so it can be loaded directly by the Flask app:

```
import pickle

with open('rdf.pkl', 'wb') as f:
    pickle.dump(model, f)
```

```
Accuracy: 0.7886178861788617
              precision    recall  f1-score   support

         0           0.88        0.44        0.58         43
         1           0.78        0.97        0.86         80

 accuracy                   0.79         123
 macro avg           0.83        0.70        0.72         123
 weighted avg        0.81        0.79        0.77         123
```

Figure 3: Model comparison / accuracy and classification report

Milestone 3: Flask Backend Development

- **Activity 3.1:** Set up app.py to load the trained model once at startup and define three routes: / (home), /predict (intake form), and /submit (POST target that returns the prediction).

```
MODEL_PATH = "model/rdf.pkl"
with open(MODEL_PATH, "rb") as f:
    model = pickle.load(f)

@app.route("/")
def home():
    return render_template("home.html")

@app.route("/predict")
def predict():
    return render_template("predict.html")
```

- **Activity 3.2:** Encode the incoming form fields with the same maps used during training, so the live request matches the training-time encoding exactly:

```
GENDER_MAP = {"Male": 1, "Female": 0}
MARRIED_MAP = {"Yes": 1, "No": 0}
DEPENDENTS_MAP = {"0": 0, "1": 1, "2": 2, "3+": 3}
EDUCATION_MAP = {"Graduate": 1, "Not Graduate": 0}
SELF_EMPLOYED_MAP = {"Yes": 1, "No": 0}
PROPERTY_AREA_MAP = {"Rural": 0, "Semiurban": 1, "Urban": 2}
```

- **Activity 3.3:** In the /submit route, read all eleven fields from request.form, build the feature vector in the exact column order the model was trained on, and call model.predict():

```
features = [
    GENDER_MAP.get(gender, 0), MARRIED_MAP.get(married, 0),
    DEPENDENTS_MAP.get(dependents, 0), EDUCATION_MAP.get(education, 0),
    SELF_EMPLOYED_MAP.get(self_employed, 0), applicant_income,
    coapplicant_income, loan_amount, loan_amount_term,
    credit_history, PROPERTY_AREA_MAP.get(property_area, 0),
]
prediction = model.predict(np.array(features).reshape(1, -1))
approved = int(prediction[0]) == 1
```

- **Activity 3.4:** Build a headline, message, and a summary dictionary of the submitted values, and pass all of it to submit.html for display.

Milestone 4: Frontend Development

- **Activity 4.1:** home.html — landing page. Introduces Smart Lender as a “Loan Approval Ledger” and lists the four groups of inputs the model uses (Applicant Profile, Income & Terms, Credit Standing, Property Area), with a button linking to /predict.

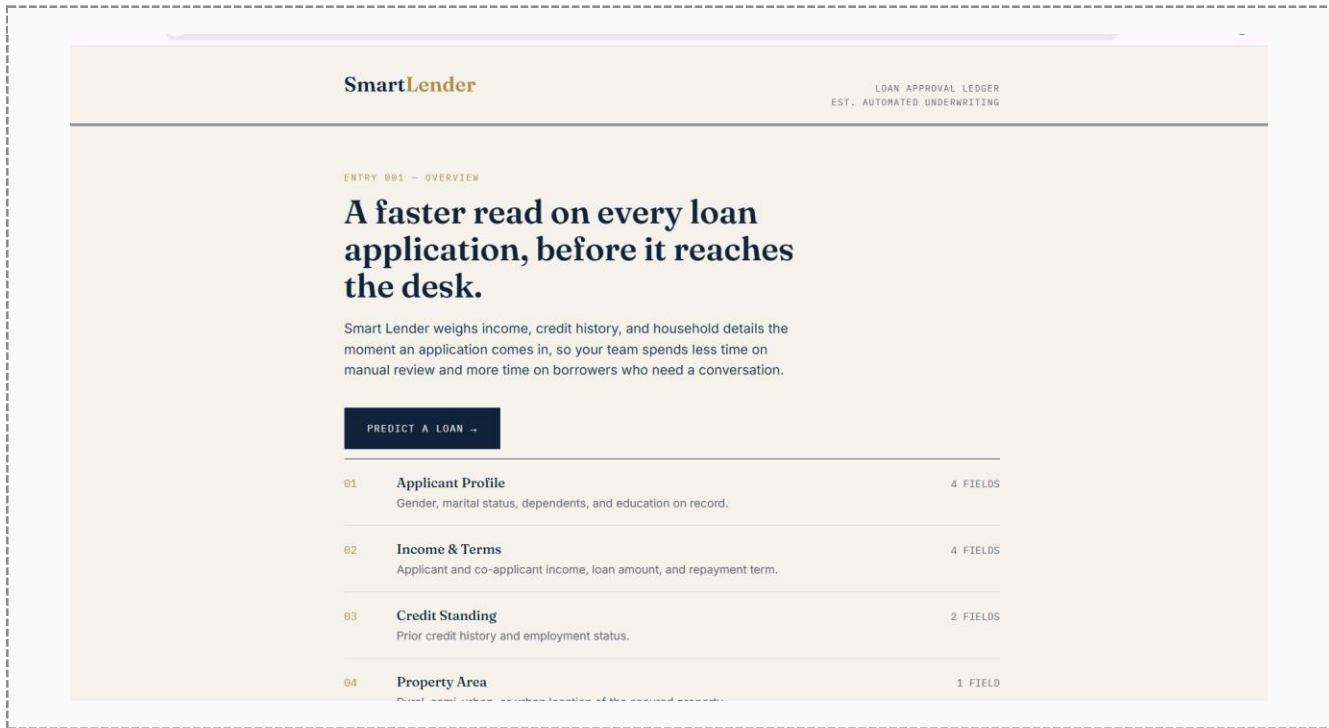


Figure 4: Smart Lender home page

- **Activity 4.2:** predict.html — intake form. A form with all eleven fields (Gender, Married, Dependents, Education, Self_Employed, ApplicantIncome, CoapplicantIncome, LoanAmount, Loan_Amount_Term, Credit_History, Property_Area) that POSTs to /submit.

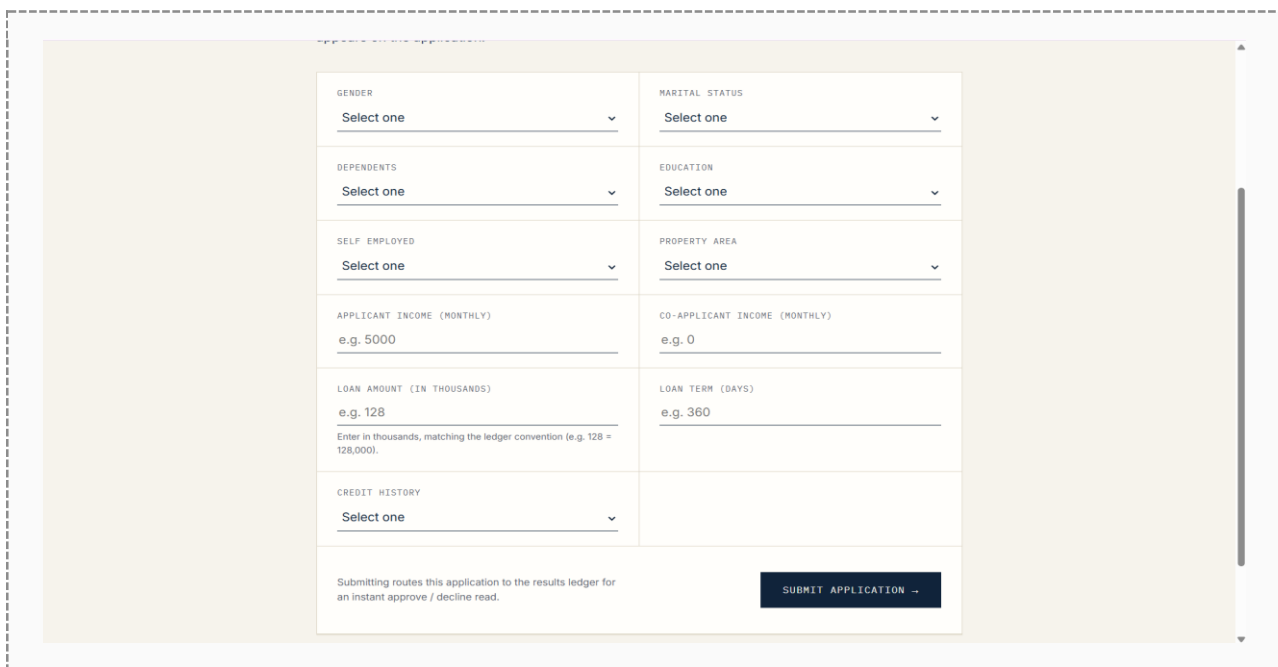

 The screenshot shows the intake form for the loan application. It is a two-column grid of form fields. The first column contains: GENDER (dropdown), DEPENDENTS (dropdown), SELF EMPLOYED (dropdown), APPLICANT INCOME (MONTHLY) (text input with example 'e.g. 5000'), LOAN AMOUNT (IN THOUSANDS) (text input with example 'e.g. 128' and a note 'Enter in thousands, matching the ledger convention (e.g. 128 = 128,000)'), and CREDIT HISTORY (dropdown). The second column contains: MARITAL STATUS (dropdown), EDUCATION (dropdown), PROPERTY AREA (dropdown), CO-APPLICANT INCOME (MONTHLY) (text input with example 'e.g. 0'), and LOAN TERM (DAYS) (text input with example 'e.g. 360'). At the bottom, there is a note 'Submitting routes this application to the results ledger for an instant approve / decline read.' and a 'SUBMIT APPLICATION' button.

Figure 5: Loan application intake form



- **Activity 4.3:** submit.html — result page. Displays a stamp-style Approved/Declined result, a headline and message, and a summary table that echoes back every field the applicant submitted.

GENDER	Male
MARITAL STATUS	Yes
DEPENDENTS	0
EDUCATION	Graduate
SELF EMPLOYED	No
APPLICANT INCOME	8000.0
CO-APPLICANT INCOME	2000.0
LOAN AMOUNT (THOUSANDS)	100.0
LOAN TERM (DAYS)	360.0
CREDIT HISTORY	Yes
PROPERTY AREA	Semiurban

Figure 6: Approved result page

GENDER	Male
MARITAL STATUS	Yes
DEPENDENTS	1
EDUCATION	Graduate
SELF EMPLOYED	Yes
APPLICANT INCOME	150000.0
CO-APPLICANT INCOME	50000.0

Figure 7: Declined result page



Activity 4.4: style.css — shared styling across all three pages: the ledger header/brand, form layout, buttons, and the approved/declined stamp colors.

Milestone 5: Testing and Deployment

- **Activity 5.1:** Test the application end-to-end with several applicant profiles (strong profile, weak profile, edge cases like zero co-applicant income or 3+ dependents) and confirm the prediction and summary table are correct.

Test Case	Key Inputs	Expected Result
Strong applicant	Good income, Credit_History = 1, Urban property	Approved
Weak applicant	No credit history, high loan amount, low income	Declined
Edge case	CoapplicantIncome = 0, Dependents = 3+	Runs without error

- **Activity 5.2:** Run the app locally with python app.py (Flask debug server) and verify all three routes (/ , /predict , /submit) work as expected.

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\kirth\Downloads\smart_lender> cd smart_lender
PS C:\Users\kirth\Downloads\smart_lender\smart_lender> python app.py
C:\Users\kirth\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages\sklearn\base.py:525: Inconsi
stentVersionWarning: Trying to unpickle estimator DecisionTreeClassifier from version 1.6.1 when usin
g version 1.9.0. This might lead to breaking code or invalid results. Use at your own risk. For more
info please refer to:
https://scikit-learn.org/stable/model_persistence.html#security-maintainability-limitations
warnings.warn(
C:\Users\kirth\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages\sklearn\base.py:525: Inconsi
stentVersionWarning: Trying to unpickle estimator RandomForestClassifier from version 1.6.1 when usin
g version 1.9.0. This might lead to breaking code or invalid results. Use at your own risk. For more
info please refer to:
https://scikit-learn.org/stable/model_persistence.html#security-maintainability-limitations
warnings.warn(
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSG
I server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
C:\Users\kirth\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages\sklearn\base.py:525: Inconsi
stentVersionWarning: Trying to unpickle estimator DecisionTreeClassifier from version 1.6.1 when usin
g version 1.9.0. This might lead to breaking code or invalid results. Use at your own risk. For more
info please refer to:
https://scikit-learn.org/stable/model_persistence.html#security-maintainability-limitations
warnings.warn(
C:\Users\kirth\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages\sklearn\base.py:525: Inconsi
stentVersionWarning: Trying to unpickle estimator RandomForestClassifier from version 1.6.1 when usin
g version 1.9.0. This might lead to breaking code or invalid results. Use at your own risk. For more
info please refer to:
https://scikit-learn.org/stable/model_persistence.html#security-maintainability-limitations
warnings.warn(
* Debugger is active!
* Debugger PIN: 377-788-583
```

Figure 8: Terminal running the Flask development server



- **Activity 5.3:** (Optional) Deploy the app publicly — e.g. with Render, Railway, PythonAnywhere, or ngrok for a temporary public URL — and note the live link in this document.

Milestone 6: Documentation and Conclusion

Smart Lender demonstrates an end-to-end applied Machine Learning workflow: collecting and cleaning a real-world loan dataset, training and comparing classification models, selecting a Random Forest model, and deploying it behind a Flask web application with a clean, purpose-built interface. By automating the first pass of loan underwriting, the system reduces manual review time and gives loan officers a consistent, data-driven starting point for their decisions, while still leaving room for manual review on borderline cases.

Appendix: Screenshot Checklist

Use this list to track which screenshots still need to be pasted into the document above.

Figure	What to capture	Where to get it
Figure 1	Architecture diagram (Browser → Flask → Model → Result)	Draw.io / PowerPoint, export as image
Figure 2	df.head() / df.info() output	Colab notebook
Figure 3	Accuracy + classification report	Colab notebook
Figure 4	Home page	Browser, localhost:5000/
Figure 5	Predict form	Browser, localhost:5000/predict
Figure 6	Result page — Approved	Browser, after submitting a strong profile
Figure 7	Result page — Declined	Browser, after submitting a weak profile
Figure 8	Terminal running Flask server	Terminal after `python app.py`

To paste a screenshot: click inside the dashed box in Word where you want it, delete the placeholder text, then Insert → Pictures → This Device and select your screenshot. Resize it to roughly the width of the page so it lines up with the caption underneath it.